DNS Security

Assignment 2 — Build, Attack, and Defend a DNS Hierarchy

CS6008: Network Security
Department of Computer Science & Engineering, IIT Bombay

Bathula Rithvik (Roll No. 23B0939)

October 23, 2026

Contents

Overview **2**

1 Phase 1 — Building the DNS Hierarchy **2**
1.1 Design . . . 2
1.2 Verification . . . 2
1.3 Report answers . . . 2

2 Phase 2 — On-Path Response Forgery **3**
2.1 Design . . . 3
2.2 Verification (2.2) — transparent relay . . . 4
2.3 Report answers . . . 4

3 Phase 3 — DNSSEC Chain of Trust **5**
3.1 Design . . . 5
3.2 Verification (3.2) — a healthy signed hierarchy . . . 5
3.3 Report answers . . . 5

4 Phase 4 — Signature Stripping & Downgrade **6**
4.1 Design . . . 6
4.2 Report answers . . . 6

5 Phase 5 — Zone Walking **7**
5.1 Design . . . 7
5.2 Verification (5.2) . . . 7
5.3 Report answers . . . 7

6 Phase 6 — Last-Mile Response Forgery **8**
6.1 Design . . . 8
6.2 Report answers . . . 8

Overview

This report documents a seven-phase study of DNS security carried out in a self-contained Docker lab. We first stand up a plain hierarchical DNS service across three isolated subnets, then mount an on-path forgery attack, add a DNSSEC chain of trust and re-run the attack, attempt a signature-stripping downgrade, walk a signed zone, move the adversary to the client's last mile, and finally encrypt that last mile with DoT. Each phase reports design choices, verification and attack evidence (terminal output and annotated packet captures), and answers to the phase's questions — not only *what* happened, but *why*.

All nodes run on isolated Docker bridge networks; nothing touches the real Internet. Every attack program is written in C/C++ with `libpcap`; `dig`, `delv`, `tshark`, and `curl` are used only to verify results. BIND 9.20.30 runs on every nameserver and on the resolver.

1. Phase 1 — Building the DNS Hierarchy

1.1 Design

The lab uses three Docker bridges — client `172.16.1.0/24`, resolver `172.16.2.0/24`, and authoritative `172.16.3.0/24` — with IP masquerade disabled on each. Docker's built-in inter-bridge isolation then drops any packet that tries to cross directly between subnets, so the *only* path between two subnets is a router container that has an interface in both. The two routers (`edge-router`, `core-router`) enable `net.ipv4.ip_forward=1` with no NAT; every other node installs static `/24` routes pointing at the router on its segment. Because a `/24` route is more specific than Docker's default route via the bridge gateway, cross-subnet traffic is forced onto our routers.

The hierarchy is a faithful miniature of the real DNS: `root-ns` is authoritative for the root and delegates `com` to `tld-ns` (glue `ns.com A 172.16.3.20`); `tld-ns` delegates `example.com` to `domain-ns` (glue `ns.example.com A 172.16.3.30`); `domain-ns` serves `A` records for `example.com` and `www.example.com` pointing at the webserver (`172.16.1.20`). The resolver is recursive with `dnssec-validation no` and uses a root-hints file that names only our private `root-ns`.

1.2 Verification

Resolution and fetch. From the client, `dig` returns the webserver address and `curl` returns the legitimate page (full log: `phase1/evidence/p1_resolve_fetch.txt`):

```
$ docker exec client dig +short example.com
172.16.1.20
$ docker exec client dig +short www.example.com
172.16.1.20
$ docker exec client curl -s http://example.com | grep -i legitimate
   <p>This is the <strong>legitimate</strong> site, served by webserver at 172.16.1.20.</p>
```

Isolation. A query sent straight to the authoritative subnet gets no answer, because the client has no route into `172.16.3.0/24` — it falls back to Docker's default gateway, which does not forward between the isolated bridges:

```
$ docker exec client dig +time=3 +tries=1 @172.16.3.10 example.com
;; communications error ... no servers could be reached
```

1.3 Report answers

(a) Root → TLD → Domain referral — what is visible on the wire. Captured on `core-router` (resolver-subnet interface) during one cold-cache resolution (`phase1/evidence/p1_referral.pcap`, decoded below):

```
1  172.16.2.10 → 172.16.3.10  query  A example.com
3  172.16.3.10 → 172.16.2.10  resp   referral: com NS ns.com, glue A 172.16.3.20
4  172.16.2.10 → 172.16.3.20  query  A example.com
6  172.16.3.20 → 172.16.2.10  resp   referral: example.com NS ns.example.com, glue A 172.16.3.30
7  172.16.2.10 → 172.16.3.30  query  A example.com
8  172.16.3.30 → 172.16.2.10  resp   ANSWER: example.com A 172.16.1.20
```

Every message is plain, unauthenticated UDP/53. On the wire an observer sees, in cleartext: the full query name (`example.com`) at every tier, the 16-bit DNS transaction ID and UDP source port that tie a reply to its query, the referral NS names and their glue addresses, and finally the answer `A 172.16.1.20`. Nothing is signed or encrypted; there is no field that binds a reply to the genuine server beyond the ID/port pair, both of which are visible to anyone on the path.

(b) How the resolver's query reaches `root-ns`. The resolver has no interface on the authoritative subnet, so it uses the installed static route:

```
$ docker exec resolver ip route get 172.16.3.10
172.16.3.10 via 172.16.2.3 dev eth0 src 172.16.2.10
```

The next hop `172.16.2.3` is `core-router`, which has a foot in both the resolver and authoritative subnets and forwards the packet (`ip_forward=1`, no NAT) out onto `172.16.3.0/24` to `root-ns`. The reply returns by the symmetric path. This is why the capture on `core-router` sees the entire referral chain.

(c) What a node on the resolver's path could do. Because the whole exchange is cleartext and unauthenticated, any node that can see and relay the resolver's traffic — exactly the position `core-router` occupies, and the position the attacker will be given in Phase 2 — can read every query and, crucially, *forge a reply*: it knows the transaction ID, the source port, and the question, so it can craft a response that the resolver will accept as genuine and cache. It can do this for one chosen name while relaying everything else untouched. We exploit precisely this in Phase 2.

2. Phase 2 — On-Path Response Forgery

2.1 Design

The adversary is placed on the resolver subnet at `172.16.2.100` and the client's rogue server `rogue-web` at `172.16.1.30`. The resolver's inbound replies are diverted through the attacker by a single route on *core-router* only (`diversion/divert-on.sh`):

```
ip route replace 172.16.2.10/32 via 172.16.2.100   # on core-router
```

Every reply bound for the resolver is therefore handed to the attacker, which is now the sole deliverer of answers to the resolver (there is no packet race — the genuine reply physically reaches us, not the resolver).

The interceptor (`attack/interceptor.c`, C + libpcap) captures each diverted reply, and in `forge` mode rewrites *only* the `A` record whose owner name is `example.com` in the answer section to `172.16.1.30`; it then recomputes the IP and UDP checksums, rewrites the Ethernet destination to the resolver's MAC, and `pcap_inject`s the frame onward. All other traffic — the root and com referrals, glue, and `www.example.com` — is relayed byte-for-byte.

Two design points were essential. First, every non-router node is pinned to `net.ipv4.ip_forward=0`: Docker containers otherwise inherit `ip_forward=1` from the host VM, and the attacker's own kernel would forward the genuine reply to the resolver before our forged one, defeating the attack and violating the "only routers forward" policy. Second, the interceptor must recompute checksums on *every* injected frame, because frames captured on the Docker veth path carry offloaded (incomplete) UDP checksums that are invalid once injected as raw bytes.

2.2 Verification (2.2) — transparent relay

With forging disabled the interceptor relays every reply unchanged, so resolution is unaffected (`phase2/evidence/p2_verify_relay.txt`):

```
$ docker exec client dig +short example.com
172.16.1.20
$ docker exec client curl -s http://example.com | grep -i legitimate
   <p>This is the legitimate site, served by webserver at 172.16.1.20.</p>
[relay] relayed 117-byte DNS reply → resolver (seen=4 forged=0)
```

This proves the attacker is genuinely on-path and that the diversion itself does not break DNS.

2.3 Report answers

(a) What the resolver cached and what the fetch returned. With forging enabled, the client resolves the poisoned name and fetches the rogue page, while the non-target name is unaffected (`phase2/evidence/p2_attack.txt`, `p2_nontarget.txt`):

```
$ docker exec client dig +short example.com         # TARGET
172.16.1.30
$ docker exec client dig +short www.example.com      # non-target
172.16.1.20
$ docker exec client curl -s http://example.com | grep DEFACED
  <h1> example.com has been DEFACED </h1>
```

The resolver's cache dump confirms the poisoned record was accepted as an answer (`phase2/evidence/p2_cache.t`

```
example.com.    3589  NS  ns.example.com.
; authanswer
                3589  A   172.16.1.30
www.example.com. 3589 A   172.16.1.20
```

The annotated capture on the attacker's NIC shows the interception and injection of the single answer (`phase2/evidence/p2_attack_decode.txt`):

```
frame  eth.src      ip.src        dns.a                      note
  7    core-router  172.16.3.30   172.16.1.20,172.16.3.30    GENUINE domain answer IN
  8    attacker     172.16.3.30   172.16.1.30,172.16.3.30    FORGED answer OUT → resolver
```

The genuine `A=172.16.1.20` (frame 7) is rewritten to `172.16.1.30` (frame 8) and delivered to the resolver; the root/com referrals that precede it are relayed unchanged.

(b) Why the attack succeeds; the value of the on-path position. Plain DNS answers are *unauthenticated*: the resolver accepts a reply solely because it matches the outstanding query's transaction ID, UDP source port, and question — none of which prove the reply came from the genuine nameserver. An *off-path* attacker who cannot see the query must *guess* those 16-bit fields (and win a race against the real reply), which is slow and unreliable. Our *on-path* position removes all of that: because we see (indeed, carry) the genuine reply, we know the ID, port, and question exactly, and we simply edit the real answer in flight. There is no guessing and no race — the resolver's only copy of the reply is the one we hand it, so acceptance is guaranteed.

(c) How non-target names stay untouched. The rewrite is gated on an exact owner-name match: the interceptor decompresses each answer RR's owner name and rewrites the RDATA only when it equals `example.com` and the type is `A`. Referrals (answer section empty), glue records (owners `ns.com`/`ns.example.com`), and `www.example.com` never match, so they are copied through verbatim. The interceptor log during a mixed run shows every `www.example.com` reply relayed with `forged=0` and only the `example.com` answer marked `REWRITTEN` (`phase2/evidence/p2_nontarget.txt`).

3. Phase 3 — DNSSEC Chain of Trust

3.1 Design

Each zone is signed with **ECDSAP256SHA256 (algorithm 13)** by `sign/sign-all.sh`, which works child-first so each child's DS can be published in its parent: `example.com` is signed and its DS placed in `com`; `com` is signed and its DS placed in the root; the root is signed last. Each authoritative server loads its `db.*.signed` file. The resolver enables `dnssec-validation yes` and installs *only* the root KSK as a static trust anchor (`config/resolver/trust-anchor.conf`); its root hints are unchanged, and the attacker, diversion route, and `interceptor.c` are identical to Phase 2.

Generated key material and signed zones are never committed — they are reproduced by the signing script. The DS chain produced on one run was:

```
com.         IN DS 53993 13 2 0291B1A7EC52...527C370F      (published in root)
example.com. IN DS 365   13 2 1D9F6FA9DDC6...CF9C717A      (published in com)
trust anchor: . static-key 257 3 13 "/a1Fphw+kLLcqF1+gJxLkPcP..."   (root KSK)
```

3.2 Verification (3.2) — a healthy signed hierarchy

With no interceptor in the path, the answer is authenticated end to end (`phase3/evidence/p3_verify.txt`):

```
$ dig +dnssec example.com
;; flags: qr rd ra ad; ...                 <- the 'ad' (authenticated-data) flag is set
example.com. 3600 IN A     172.16.1.20
example.com. 3600 IN RRSIG A 13 2 3600 ... 19890 example.com. TVZej6fd...

$ delv -a trust-anchor.conf @172.16.2.10 example.com
; fully validated                          <- chain root KSK → com DS → example.com DS → A
```

3.3 Report answers

(a) What the resolver does with the forged response; how it differs from Phase 2; what it checks. The Phase 2 attack is re-run *unchanged*. The interceptor still rewrites the `A` record to `172.16.1.30`, but the outcome is now the opposite of Phase 2 (`phase3/evidence/p3_attack.txt`):

```
$ dig example.com
;; →>HEADER<<- status: SERVFAIL ; ANSWER: 0
$ dig +short example.com
(empty)                          # the client gets nothing; curl fetches no page
[forge] ... example.com A REWRITTEN → rogue-web   # the attacker is still forging
```

The resolver's own log says exactly what it checked and why it refused the answer:

```
validating example.com/A: no valid signature found
RRSIG failed to verify resolving 'example.com/A/IN': 172.16.3.30#53
```

This is the difference from Phase 2. There, the resolver accepted any syntactically valid reply that matched the query's ID/port/question. Here it additionally requires an `RRSIG` over the `A` RRset that verifies under a `DNSKEY` which is itself vouched for by a `DS` in the parent, up to the trust anchor. The attacker can change the `A` to `172.16.1.30` but *cannot* produce a matching signature — it holds no private key — so the RRSIG that arrives still signs `172.16.1.20`. Signature verification over the tampered RRset fails and the resolver returns SERVFAIL rather than serve unauthenticated data. The capture (`p3_attack_decode.txt`) shows the forged `A=172.16.1.30` travelling with that stale RRSIG.

(b) Compared to Phase 1 on the wire: what can the attacker still read, and what does DNSSEC protect? Nothing on the wire became secret. DNSSEC *signs* records; it does not *encrypt*

them. The capture shows every query name, every answer (including the genuine `172.16.1.20` and the attacker's `172.16.1.30`), the referral chain, transaction IDs and ports — all still in cleartext, exactly as in Phase 1. DNSSEC therefore provides *origin authentication and integrity* of DNS data (a resolver can tell a tampered or spoofed record from a genuine one), but it provides *no confidentiality*: an on-path observer still learns what was asked and what the true answer is, and can still see (though no longer successfully alter) the exchange. It also does not protect the last hop from resolver to a non-validating stub (Phase 6), nor hide which names exist (Phase 5).

(c) Signing-algorithm choice and private keys. We chose **ECDSAP256SHA256 (alg 13)**: its 256-bit keys and signatures are far smaller than equivalent-strength RSA (RSASHA256 would need ≥2048-bit keys), which keeps responses small — useful against UDP fragmentation and amplification — while validation stays fast; it is also the modern recommendation for new deployments. The trade-off is that ECDSA verification is slightly more CPU-intensive per operation than RSA and demands a sound random source at signing time. **No private key belongs in the submitted repository**: the `K*.private` files and the signed zones are regenerated by `sign/sign-all.sh`, and only the (public) trust anchor is needed by the resolver, itself regenerated by the script. Committing a zone-signing private key would let anyone forge valid signatures for the zone, destroying the very guarantee DNSSEC provides.

4. Phase 4 — Signature Stripping & Downgrade

4.1 Design

The Phase 3 signed hierarchy and validating resolver are reused unchanged. The interceptor gains a `strip` mode: for `example.com` responses it re-emits the DNS message dropping the zone's own-data security — the `A`'s RRSIG, the `DNSKEY`, and the `DS` in the com referral — and rewrites the `A` to `172.16.1.30`. Crucially it *keeps* any `NSEC`/`NSEC3` record and the RRSIGs that cover them: those are the *parent's* authenticated statement about whether a secure delegation exists, which the attacker cannot forge. The signing script takes a mode: `full` publishes `example.com`'s DS in `com` (the Phase-3 chain); `nods` omits it (the child is signed but its DS was never given to the parent).

4.2 Report answers

(a) Run A — stripping against the fully signed chain. With the DS present, stripping the signatures and even the DS fails (`phase4/evidence/p4_runA.txt`):

```
[STRIPPED example.com DNSSEC RRs (DS/RRSIG/DNSKEY)]   # attacker removes them on the wire
[STRIPPED example.com sigs + FORGED A]                # and injects A=172.16.1.30
$ dig example.com   →  status: SERVFAIL     (dig +short: empty)
resolver log:  broken trust chain resolving 'example.com/A/IN'
```

The attacker *can* delete bytes on the wire, but it *cannot make the zone look legitimately unsigned.* The reason `example.com` is treated as secure is the `DS` record published in `com` — and `com` is signed. To convince the resolver that `example.com` is unsigned, the attacker would have to present `com`'s *signed* statement that no DS exists (a signed NSEC), which requires `com`'s private key. Stripping the DS on the wire just yields an *unsigned* "no DS" that the root-anchored resolver will not believe: it still holds a secure delegation into `example.com`, finds no valid signature to complete the chain, and returns SERVFAIL. Downgrade is resisted.

(b) Run B — the single change that reopens it. The only change is in the *parent*: `com` publishes **no DS** for `example.com` (the child stays fully signed). Now `com`'s signed `NSEC` *authentically* proves that `example.com` has no DS, so the resolver concludes the delegation is *insecure*, stops requiring signatures for it, and accepts the forged record (`phase4/evidence/p4_runB.txt`):

```
# baseline (no attacker): dig example.com → 172.16.1.20, flags: qr rd ra   (no 'ad' → insecure)
# attack (strip+forge):
```

```
$ dig example.com            → status: NOERROR, flags: qr rd ra   (no 'ad')
$ dig +short example.com     → 172.16.1.30
$ curl http://example.com    → DEFACED page
```

The same strip-and-forge that failed in Run A now lands the client on the rogue page, because the resolver never had a secure link into `example.com` to begin with.

(c) Where downgrade resistance comes from, and the operator mistake that forfeits it. Downgrade resistance does not come from the child signing its zone — in both runs `example.com` is fully signed. It comes from the **authenticated DS in the parent**: the signed `DS` is what tells a validator "this delegation is secure, demand signatures," and its *signed absence* (NSEC) is what tells it "this delegation is insecure." Because both the presence and the absence are signed by the parent, an on-path attacker — who holds no keys — can neither invent a secure delegation nor tear one down. The exact operator mistake that forfeits this protection is the one in Run B: **signing the child but never publishing its DS in the parent**. The zone looks protected (it has keys and RRSIGs) but is globally treated as unsigned, so the plain Phase-2 forgery works again. Signing a zone is only half the job; the DS must reach the parent.

5. Phase 5 — Zone Walking

5.1 Design

The Phase 3 signed, validating deployment is reused. `example.com` is populated with six hidden internal-service names — `dev, git, mail, vpn, backup, monitoring` — that are not advertised anywhere, alongside the public apex and `www`. The zone is signed with **NSEC** (the `dnssec-signzone` default), and the fix re-signs it with **NSEC3**. The walker (`attack/zonewalk.c`) crafts every query and reads every reply with libpcap — no resolver library, no AXFR, no wordlist — sending to the recursive resolver, which fetches and returns the signed denials.

5.2 Verification (5.2)

A query for a name that does not exist is answered with a signed NSEC that, to prove the gap, names the *existing* records on either side (`phase5/evidence/p5_verify.txt`):

```
$ dig +dnssec nonexistent.example.com      →  status: NXDOMAIN
example.com. 3600 IN NSEC backup.example.com. A NS SOA RRSIG NSEC DNSKEY
```

The denial for `nonexistent` already leaks that `backup.example.com` exists.

5.3 Report answers

(a) The names recovered, and the exact queries that produced them. Starting from `example.com` alone, the walker follows the NSEC chain, learning one new name per step (`phase5/evidence/p5_wal`

```
query 1  NSEC example.com            → backup.example.com
query 2  NSEC backup.example.com     → dev.example.com
query 3  NSEC dev.example.com        → git.example.com
query 4  NSEC git.example.com        → mail.example.com
query 5  NSEC mail.example.com       → monitoring.example.com
query 6  NSEC monitoring.example.com → ns.example.com
query 7  NSEC ns.example.com         → vpn.example.com
query 8  NSEC vpn.example.com        → www.example.com
query 9  NSEC www.example.com        → example.com     (wraps to apex: done)

Recovered 8 names in 9 queries: backup, dev, git, mail, monitoring, ns, vpn, www
```

Each step queries the immediate canonical successor of the current name (a nonexistent `\000.name`, with the EDNS DO bit set) so the authoritative `example.com` NSEC covering that gap is returned; its *next* field is the next existing name. Nine queries recover all eight names — within the "number of names +2" budget. All six hidden services are exposed.

(b) Which configuration made the walk possible, and why signed denial leaks what plain DNS does not. NSEC made it possible. A signed proof of non-existence must be computed *before* the query is known (it is signed offline), so it cannot be about the queried name itself — instead each NSEC asserts "nothing exists between name X and name Y," naming two real adjacent names. Chaining those intervals is exactly a sorted listing of the zone. Plain (unsigned) DNS leaks nothing comparable: it answers a nonexistent name with a bare NXDOMAIN and no information about neighbours, because it has no need to prove anything. DNSSEC's requirement to *prove* non-existence, offline and in advance, is precisely what forces it to reveal the surrounding names.

(c) What the fix costs, and what an enumerator can still learn. Re-signing with NSEC3 closes the online walk: the same tool now recovers *zero* names, while a nonexistent name still gets an authenticated NXDOMAIN — proven by an NSEC3 record whose owner is a *hash* of the name (salt `DEADBEEF`), with no readable successor to follow (`phase5/evidence/p5_walk_nsec3_fails.txt`):

```
nonexistent.example.com →  NXDOMAIN, proven by  <hash>.example.com NSEC3 1 0 0 DEADBEEF <hash> ...
zonewalk →  Recovered 0 names
```

The cost to the operator is real: NSEC3 hashing (and optionally many iterations) makes signing and denial responses more expensive, and it adds the NSEC3PARAM/salt machinery. And it is not a complete cure: NSEC3 publishes the *hashes* of every name, so an enumerator can still harvest the full set of hashes by walking the NSEC3 chain and then mount an *offline* dictionary/brute-force attack against them (tools such as `nsec3walker`/`hashcat`). NSEC3 raises the cost of enumeration from a handful of online queries to an offline cracking effort; it does not make the names secret. Only not signing (or using aggressive-NSEC/online-signing designs) changes that trade-off further.

6. Phase 6 — Last-Mile Response Forgery

6.1 Design

Upstream (Phases 3–5) is untouched and still validating. The attacker is relocated to `172.16.1.100` on the client subnet, and `edge-router` diverts the resolver's client-bound replies (for `172.16.1.10` and `.11`) through it — a route on the router only, no ARP spoofing. The interceptor binary is *unchanged* from Phase 2/3; it is simply pointed at a client as its delivery target. Two clients are used: **Client A** (`client`, an ordinary stub using `dig`/`curl`) and **Client B** (`client-b`, a locally-validating stub using `delv` with the root key as its own trust anchor).

6.2 Report answers

(a) Outcome for each client. With forging on (`phase6/evidence/p6_attack.txt`):

```
Client A (dig/curl):  example.com → 172.16.1.30,  flags: qr rd ra ad,  curl → DEFACED   (POISONED
    )
Client B (delv):      ;; validating example.com/A: no valid signature found
                      ;; resolution failed: RRSIG failed to verify                      (CAUGHT)
```

Client A lands on the rogue page; Client B rejects the answer.

(b) DNSSEC validates upstream — why does it not stop this? Where validation happens vs. where trust is placed. DNSSEC validation happened, correctly, at the *resolver*: the resolver fetched and verified the signatures for `example.com` on the resolver↔authoritative path,

and its genuine reply to the client even carried `A=172.16.1.20` with the `ad` flag. But validation happening *there* only protects *that* hop. The last hop — resolver→client — is plain, unauthenticated DNS, and that is where the attacker now sits. The distinction is between *where validation happens* (at the resolver, upstream) and *where trust is placed* (at the stub, which trusts its resolver's word). Client A places its trust in a link DNSSEC never covered, so the attacker who controls that link defeats it despite a perfectly valid chain one hop away.

(c) What the `ad` flag means, why the attacker can set it, and what Client B does differently. The `ad` (authenticated-data) bit in a reply is the resolver's *claim* "I validated this." It is a single bit in the cleartext DNS header, protected by nothing on the last mile. Our capture shows the forged reply carrying `A=172.16.1.30` *with* `ad=1` (`phase6/evidence/p6_adflag.txt`): the attacker rewrote the answer and left the bit set, so Client A, which believes the bit, is told a forged record was "authenticated." An on-path attacker can set that bit to any value at will. Client B does something fundamentally different: it does not believe anyone's `ad` bit. Holding the root key as its own trust anchor, it fetches the `DNSKEY`/`RRSIG` records and *re-verifies the signatures itself.* The forged `A=172.16.1.30` has no RRSIG that verifies (the genuine RRSIG signs `172.16.1.20`), so Client B reports "RRSIG failed to verify" and refuses the answer. The lesson: DNSSEC's guarantee reaches only as far as validation is actually performed; to be safe on an untrusted last mile the endpoint must validate locally (or the link must be authenticated — Phase 7).